

CASA Evidence — Principle of Least Privilege

Question: Explain / provide documentation that the principle of least privilege exists — users should only be able to access functions, data, files, URLs, controllers, services, and other resources for which they possess specific authorization.

Application: Provenance (Next.js App Router monorepo on the Makerkit "lite" SaaS starter, backed by Supabase — Postgres + GoTrue Auth + Storage — plus Stripe for billing).

This document is a companion to [authentication-and-access-control.md](#). That document explains *how* identity, roles, and access-control checks work end-to-end. This document focuses specifically on evidence that access is **scoped to the minimum necessary** at every layer — database privileges, row data, server functions, admin surfaces, and machine-to-machine credentials — rather than granted broadly and merely hidden by the UI.

1. Database layer: privileges are denied by default, then opened narrowly

The schema starts from **zero privilege** and every grant afterward is narrow and explicit — least privilege is the default posture, not an afterthought.

[20241219010757_schema.sql](#) ("Revoke default privileges from public schema"), applied before a single table is created:

```
alter default privileges revoke execute on functions from public;
revoke all on schema public from public;
revoke all privileges on database "postgres" from "anon";
revoke all privileges on schema "public" from "anon";
revoke all privileges on schema "storage" from "anon";
revoke all privileges on all sequences in schema "public" from "anon";
revoke all privileges on all functions in schema "public" from "anon";
revoke all privileges on all tables in schema "public" from "anon";
alter default privileges in schema public revoke execute on functions from anon, authenticated;
```

Only after this blanket revoke does the schema [grant usage on schema public to authenticated](#) and to [service_role](#). Every subsequent migration must then **explicitly** grant exactly what a role needs:

Grant pattern found in the schema	Count	What it proves
create policy ... (RLS policies)	229, across 44 migrations	Every protected table's access is itemised per action (select/insert/update/delete), not blanket
grant select on table ... to anon	13 occurrences	The only thing the public/anonymous role ever receives is read access to specific, already-curated tables (e.g. published artworks , blog_posts , open_calls) — never insert/update/delete (the one exception, open_call_submissions , is an explicit public "apply" form, still insert -only)
revoke all on function ... from public + grant execute ... to service_role only	e.g. admin_top_artwork_uploaders()	Server-only analytics functions are unreachable by anon/authenticated entirely — only the elevated backend role can call them

This means a compromised or malicious **anon/authenticated** Postgres session — e.g. via a leaked anon API key, which is a public key by design — **cannot** read or write anything beyond what an explicit grant + an explicit RLS policy both allow, even before application code runs at all.

2. Row Level Security: least privilege at the row, not just the table

Table-level grants only say "this role may run **select** on this table at all." Row Level Security (RLS) then restricts **which specific rows** are visible/writable, scoped almost universally to **auth.uid()** — i.e. a signed-in user gets exactly their own data, no more.

Representative policy set, **artworks** (20250103000000_create_artworks.sql):

```
-- Users can read their own artworks
create policy artworks_read_own on public.artworks
  for select to authenticated
  using (account_id = (select auth.uid()));

-- Public can read ONLY verified artworks (never drafts)
create policy artworks_read_public on public.artworks
  for select to anon, authenticated
  using (status = 'verified');

-- Users can insert/update/delete ONLY their own artworks
create policy artworks_insert on public.artworks
  for insert to authenticated
  with check (account_id = (select auth.uid()));
```

This pattern is repeated per-table across the schema (accounts, user_profiles, gallery_members, consignments, exhibitions, subscriptions, notifications, profile_sites, operations_* tables, etc.) — **44 migrations enable RLS**, producing the 229 policies counted above. A user who is fully authenticated and has table-level **select/update** grants still cannot read or modify another user's row, because the database itself, not the application, makes that decision on every query.

Team/role-scoped example — **gallery_members** (20250126000000_add_gallery_members.sql) narrows even *within* a shared resource by the caller's specific role on that resource:

```
create policy gallery_members_read_gallery on public.gallery_members
  for select to authenticated
  using (
    exists (
      select 1 from public.gallery_members gm
      where gm.gallery_profile_id = gallery_members.gallery_profile_id
      and gm.user_id = (select auth.uid())
      and gm.role in ('owner', 'admin')  -- plain 'member' cannot list all members
    )
    or exists ( -- the gallery's own account can always see its team
      select 1 from public.user_profiles up
      where up.id = gallery_members.gallery_profile_id
      and up.user_id = (select auth.uid()) and up.role = 'gallery'
    )
  );
```

A **member** of a gallery team can see and manage content, but the "who's on this team" view is only granted to **owner/admin** — a concrete example of scoping to "specific authorization," not just "is this person on the team at all."

Storage (files) — folder-prefix ownership, same principle applied to object storage rather than table rows:

```
create policy artworks_storage_insert on storage.objects
  for insert to authenticated
```

```

with check (
    bucket_id = 'artworks'
    and split_part(name, '/', 1) = auth.uid()::text -- can only write inside own folder
);

```

A user with **insert** privilege on the **artworks** bucket can still only write files under a path prefixed with their own **auth.uid()** — they cannot overwrite or delete another user's uploaded files even though they share the same bucket.

3. SECURITY DEFINER functions: elevated code paths still re-check the caller

A few operations legitimately need to run with elevated database privileges (e.g. writing derived/aggregate data, or bypassing a recursive RLS check). Rather than exposing broad elevated access, these are implemented as narrow **security definer** functions that **re-validate the caller inside the function body**, and pin **search_path** to prevent function-hijacking:

202605120000002_record_user_heartbeat_enforce_caller.sql — a **security definer** function that could, in principle, write *any* user's presence row, is hardened to refuse to do so for anyone but the caller themselves:

```

create or replace function public.record_user_heartbeat(p_user_id uuid)
...
security definer
set search_path = public
as $$
begin
    if (select auth.uid()) is distinct from p_user_id then
        raise exception 'not authorized' using errcode = '42501';
    end if;
    ...
end;
$$;

revoke all on function public.record_user_heartbeat(uuid) from public;
grant execute on function public.record_user_heartbeat(uuid) to authenticated;

```

202607080000004_protect_admin_flag.sql — prevents privilege *escalation* specifically: an authenticated user can update their own **accounts** row (normal RLS **accounts_update** policy), but a trigger blocks the one dangerous field:

```

create or replace function kit.protect_admin_flag()
returns trigger security definer set search_path = ''
as $$
begin
    if current_setting('role', true) = 'authenticated' then
        if (new.public_data ->> 'admin')::boolean is true
            and coalesce((old.public_data ->> 'admin')::boolean, false) is false then
            raise exception 'Unauthorized: cannot self-grant admin privileges';
        end if;
    end if;
    return new;
end;
$$;

```

i.e. a normal user's broad "update my own account" permission is explicitly *not* allowed to include granting themselves the admin role — the elevated **admin** flag can only be set via the **service_role** (a human operator acting directly against the database), never through the app's own update path.

`20260512000001_admin_top_artwork_uploaders.sql` — an admin-analytics aggregate function is granted **only** to `service_role`, with `anon/authenticated` explicitly revoked:

```
revoke all on function public.admin_top_artwork_uploaders(integer) from public;
grant execute on function public.admin_top_artwork_uploaders(integer) to service_role;
```

4. Application layer: the most powerful client is the hardest to reach

The Supabase **service-role key** bypasses RLS entirely (it is the database super-user credential). Least privilege here means: (a) very few code paths may use it, (b) it can never leave the server, and (c) every path that does use it is behind the strictest possible authorization gate.

(a) + (b) — isolated at the module level, not just by convention:

```
import 'server-only';
...
export function getServiceRoleKey() {
  return z.string({ required_error: message }).min(1, { message }).parse(
    process.env.SUPABASE_SERVICE_ROLE_KEY,
  );
}
```

`import 'server-only'` is a Next.js guard that **fails the build** if this module is ever imported into a client-side bundle — the service-role key literally cannot be shipped to a browser by accident. `SUPABASE_SERVICE_ROLE_KEY` (like `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `CRON_SECRET`) is also deliberately **not** prefixed `NEXT_PUBLIC_`, which is the Next.js convention boundary between server-only and browser-exposed environment variables — see [.env.example](#), where only non-sensitive config (`NEXT_PUBLIC_PRODUCT_NAME`, `NEXT_PUBLIC_SITE_URL`, ...) carries that prefix.

(c) — single choke point for the whole admin surface. Every page under `/admin/*` — including analytics components that call `getSupabaseServerAdminClient()` (the service-role client) to read cross-user data like `auth.admin.listUsers()` — is rendered inside one layout that enforces the full admin+MFA gate before any child component runs:

```
export default async function AdminLayout({ children }: { children: React.ReactNode }) {
  // Enforce authentication, admin status, and MFA assurance at the layout level.
  const { requiresMfaSetup, mfaGraceDeadline } = await requireAdmin();
  ...
}
```

`requireAdmin()` (`src/lib/admin.ts`) itself layers three independent checks before returning — see [authentication-and-access-control.md](#) §3.3 for the full breakdown — and **fails closed** (denies access) on any error rather than defaulting to allow. Because the gate lives in the shared layout, there is no admin sub-page or component that can be reached without passing it — a new admin feature added under `/admin/*` inherits the restriction automatically rather than needing to remember to add its own check.

Per-action guards for narrower operations — not every privileged action needs full service-role access; most just need "signed in" or "signed in + owns this row," and the codebase uses the narrowest applicable guard rather than reaching for the broadest one:

Guard	Grants	Used for
<code>requireUser(client)</code>	Any authenticated user, using their own cookie-bound (RLS-scoped) session	Regular pages/actions

Guard	Grants	Used for
<code>requireAdminApi()</code> / <code>requireAdminUser()</code>	Authenticated and <code>public_data.admin = true</code> and MFA-verified	Admin-only API routes/Server Actions
<code>getSupabaseServerAdminClient()</code> (service-role)	Full database bypass — reserved for operations that structurally cannot be expressed as "the current user's own row," e.g. cross-user admin analytics	Only inside the already-gated <code>/admin/</code> tree
Explicit ownership re-check, e.g. <code>assertArtworkOwned()</code> in <code>src/app/operations/_actions/consignments.ts</code>	Confirms <code>artworks.account_id === callerId</code> before acting, in addition to RLS	Server Actions that touch a resource by ID supplied from the client

5. Machine-to-machine access: scheduled/cron endpoints are scoped and hardened

Cron-triggered routes (`/api/cron/stripe-reconcile`, `/api/cron/operations-alerts`, `/api/cron/lifecycle-emails`) run with no interactive user session at all, so they cannot rely on `auth.uid()`-based RLS. Least privilege is instead enforced by a narrow, single-purpose shared secret, checked in **constant time** to avoid a timing side-channel that could otherwise be used to guess the secret byte-by-byte:

```
function isAuthorized(request: NextRequest) {
  const secret = process.env.CRON_SECRET;
  if (!secret) {
    console.error('[CRON/stripe-reconcile] CRON_SECRET is not set');
    return false;
  }
  const header = request.headers.get('authorization');
  if (!header?.startsWith('Bearer ')) return false;
  return constantTimeEquals(header.slice(7), secret);
}
```

Note the fail-closed default: if `CRON_SECRET` is missing from the environment, the route denies the request rather than silently allowing it. The same route additionally only constructs a Stripe client from `STRIPE_SECRET_KEY` (never exposed client-side) and, inside its own logic, only ever reconciles subscriptions it already holds a matching row for — it is not a generic "do anything to Stripe" endpoint.

Stripe **webhooks** (`src/app/api/webhooks/stripe/route.ts`) follow the same idea from the other direction: the app doesn't trust an inbound request just because it says it's from Stripe — it verifies the Stripe signature before processing any event, so an attacker who merely knows the webhook URL cannot inject fake payment/fulfillment events (see [docs/casa-evidence/payment-log-sample.md](#) for the verified log trail of that path).

6. Summary — resource-by-resource

Resource / action	Who can access	How it's enforced
Another user's artwork/profile/consignment row	Nobody but the owner (or an explicit collaborator role, e.g. gallery team)	RLS using <code>(account_id = auth.uid())</code> — table grant alone is insufficient

Resource / action	Who can access	How it's enforced
Draft (unpublished) artwork	Owner only, never <code>anon</code> /public	RLS <code>artworks_read_public</code> explicitly filters <code>status = 'verified'</code>
Another user's storage files	Nobody	Storage RLS keys off the <code>auth.uid()</code> folder prefix
Gallery team member list	Only that gallery's <code>owner/admin</code> , not plain <code>members</code>	Role check embedded directly in the RLS policy
Self-granting the <code>admin</code> flag	Nobody (fails even for an otherwise-valid "update my account" request)	<code>protect_admin_flag</code> trigger blocks the specific field transition
<code>/admin/*</code> pages & the service-role client	Only accounts with <code>public_data.admin = true</code> and a verified MFA session	Single gate in <code>src/app/admin/layout.tsx</code> → <code>requireAdmin()</code>
Admin-only aggregate DB functions (e.g. <code>admin_top_artwork_uploaders</code>)	Only the <code>service_role</code> Postgres role	<code>revoke all from public + grant execute to service_role</code>
Cron/scheduled endpoints	Only the caller holding <code>CRON_SECRET</code>	Constant-time bearer-token check, fail-closed if unset
Service-role / Stripe secret credentials	Server process only, never the browser	<code>import 'server-only'</code> guard + non- <code>NEXT_PUBLIC</code> env vars
Public/anonymous visitors	Read-only, and only rows already marked public (<code>verified</code> , <code>published</code> , etc.)	Narrow <code>grant select ... to anon</code> per table, no <code>anon</code> write grants except a single explicit public-submission form

7. Conclusion

Least privilege is implemented as a **layered, default-deny** system rather than a single check:

verb-by-verb (`select`-only to `anon` in almost every case).

caller's own rows (or their specific role on a shared resource).

internally and are granted to the narrowest possible Postgres role.

isolated to server-only modules and reachable only behind a single, fail-closed admin+MFA gate that every admin page inherits.

checked secrets rather than any broader credential.

- Postgres privileges start at zero and are opened table-by-table,
- Row Level Security then narrows *within* a granted table to the
- Elevated `security definer` functions re-validate the caller
- The most powerful application credential (service-role) is
- Machine-to-machine endpoints use single-purpose, constant-time-

Any one of these layers being misconfigured does not, by itself, expose data beyond its own scope — each layer independently enforces the same "only what you're specifically authorized for" rule.